Why Assembly?

- **The Bridge:** Assembly is the thinnest abstraction over the hardware.
- **Why NASM?**
  - Uses Intel Syntax (destination first: mov dst, src).
  - Widely used in Linux and Windows development.
  - Supports a vast range of output formats (ELF64, Mach-O, Win64).
- **NASM Philosophy:** It is "Netwide." It aims to be portable and efficient, without the heavy "auto-intelligence" of MASM/TASM.

Anatomy of a NASM Source File

- **Sections: NASM organizes code into logical segments:**
  - section **.data**: For initialized global variables (e.g., constants, strings).
  - section **.bss**: For uninitialized data (Block Started by Symbol).
  - section **.text**: Where the actual machine instructions reside.
- **Labels: Names followed by a colon (e.g., _start:) representing memory addresses.**
- **Global Directive: global _start — tells the linker the entry point.**

NASM Syntax Basics

- **Format:** `label: instruction operands ; comment`
- **Constants:**
  - Decimal: 200
  - Hex: 0xc8 or 0ach
  - Binary: 11001000b
- **Expressions: NASM allows basic math in labels (e.g., mov rax, label + 10).**
- **Case Sensitivity: Instructions are case-insensitive, but labels are case-sensitive by default.**

Toolchain: From ASM to EXE

- **Assembly: Use NASM to create an object file.**
  - nasm -f win64 program.asm -o program.o
- **Linking: Use ld (GoLink or gcc) to create the executable.**
  - ld/GoLink/gcc program.o -o program.exe
- **Execution: program.exe**
- **Debugging:** Use objdump (from mingw) to inspect the machine code.

The NASM Preprocessor

- **Power of the Preprocessor: NASM includes a very powerful preprocessor similar to C, but more flexible.**
- **Single-line Macros: %define**
  - %define CTRL_L 12
  - Used for constants and simple aliases.
- **Multi-line Macros: %macro and %endmacro**
  - Allows for "pseudo-instructions.“
  - Example: A macro to print a string can wrap multiple mov and syscall instructions.

Anatomy of a Multi-line Macro

- **Syntax:** %macro Name Number_of_Args
- **Argument Access:** Use %1, %2, etc.
- **Local Labels: Labels inside macros must start with %% to avoid name collisions when the macro is used multiple times.**
  - %%loop_start:
- **Overloading: You can define multiple macros with the same name but different argument counts.**

Conditional Assembly

- **Directives: %if, %elif, %else, %endif.**
- **Use Cases:**
  - Writing code that supports multiple Operating Systems (e.g., %ifdef LINUX).
  - Debugging blocks: %ifdef DEBUG ... %endif.
- **%assign vs. %define:**
  - %assign handles numeric expressions and can be re-assigned (useful for loop counters in macros).